

SelVeri: Self-Verifying Programming Language
High Level Language Syntax and Semantics Document
v0.9

Yusuf Demir
Yağız Kaan Aydoğdu

April 10, 2026

1 Syntax

1.1 Lexical Categories

Identifiers must start with a letter or underscore, and continue with letters, digits, or underscores. Integer and floating-point literals are given by the following lexical categories:

$$\begin{aligned}\langle Ident \rangle &::= [A-Za-z][A-Za-z0-9_]* \\ \langle IntLit \rangle &::= [0-9]^+ \\ \langle FloatLit \rangle &::= [0-9]^+.[0-9]^+\end{aligned}$$

1.2 Types

All variables and function return values are statically typed.

$$\begin{aligned}\langle BasicType \rangle &::= \text{Int} \mid \text{Float} \\ \langle Type \rangle &::= [\langle BasicType \rangle \mid \text{List}[\langle BasicType \rangle, \langle IntLit \rangle(, \langle AExp \rangle)^+] \mid \text{type}(\langle ident \rangle)]\end{aligned}$$

1.3 Immediates

$$\begin{aligned}\langle Imm \rangle &::= \langle IntLit \rangle \mid \langle FloatLit \rangle \mid \langle ListLit \rangle \\ \langle ListLit \rangle &::= [] \mid [\langle Imm \rangle(, \langle Imm \rangle)^*]\end{aligned}$$

1.4 Expressions

Arithmetic Expressions We declare the syntax of arithmetic expressions so that the operator precedence makes sense in the usual manner.

$$\begin{aligned}\langle AExp \rangle &::= \langle ASum \rangle \\ \langle ASum \rangle &::= \langle ASum \rangle + \langle ATerm \rangle \mid \langle ASum \rangle - \langle ATerm \rangle \mid \langle ATerm \rangle \\ \langle ATerm \rangle &::= \langle ATerm \rangle * \langle AFactor \rangle \mid \langle ATerm \rangle / \langle AFactor \rangle \mid \langle AFactor \rangle \\ \langle AFactor \rangle &::= - \langle AFactor \rangle \mid \langle AAtom \rangle \\ \langle AAtom \rangle &::= \langle Imm \rangle \mid \langle Ident \rangle \mid \text{len}(\langle Ident \rangle) \mid \langle IndexedVal \rangle \mid (\langle AExp \rangle) \\ \langle IndexedVal \rangle &::= \langle IndexedVal \rangle[\langle Aexp \rangle] \mid \langle Ident \rangle[\langle Aexp \rangle]\end{aligned}$$

`len` keyword defined above semantically gives us the length of a list. In the current implementation, length of a list is fixed, so it can be resolved via a lookup from the scope table. It must not be confused with a function call, as no scope switches happen while resolving this value.

Boolean Expressions Boolean expressions used in guards and specifications include arithmetic expressions as truthy values.

$$\begin{aligned}\langle BExp \rangle &::= \langle BOr \rangle \\ \langle BOr \rangle &::= \langle BOr \rangle \text{ or } \langle BXor \rangle \mid \langle BXor \rangle \\ \langle BXor \rangle &::= \langle BXor \rangle \text{ xor } \langle BAnd \rangle \mid \langle BAnd \rangle \\ \langle BAnd \rangle &::= \langle BAnd \rangle \text{ and } \langle BNot \rangle \mid \langle BNot \rangle \\ \langle BNot \rangle &::= \text{not } \langle BNot \rangle \mid \langle BAtom \rangle \\ \langle BAtom \rangle &::= \text{true} \mid \text{false} \mid \langle Comparison \rangle \mid \langle AExp \rangle^1 \mid (\langle BExp \rangle) \\ \langle Comparison \rangle &::= \langle AExp \rangle \langle CompOp \rangle \langle AExp \rangle \\ \langle CompOp \rangle &::= = \mid <= \mid < \mid >= \mid >\end{aligned}$$

¹We interpret results of arithmetic expressions as boolean values where needed using equality to zero as in other imperative languages like C. This is useful for being able to write statements like `while x`, where `x` is an integer variable.

1.5 Specifications (Runtime Assertions)

Here the syntax is also constructed in a way that establishes operator precedence rules. In the implemented parser, $\langle \text{Spec} \rangle$ nodes are not parsed further and parser accepts any character sequence in the place of this node. This is because specifications are compiled to the intermediate language literally and parsed for the verification module starting from the IR code.²

$$\begin{aligned}
\langle \text{Spec} \rangle &::= \langle \text{SpecImp} \rangle \\
\langle \text{SpecImp} \rangle &::= \langle \text{SpecOr} \rangle \Rightarrow \langle \text{SpecImp} \rangle \mid \langle \text{SpecOr} \rangle \\
\langle \text{SpecOr} \rangle &::= \langle \text{SpecOr} \rangle \parallel \langle \text{SpecAnd} \rangle \mid \langle \text{SpecAnd} \rangle \\
\langle \text{SpecAnd} \rangle &::= \langle \text{SpecAnd} \rangle \&\& \langle \text{SpecTemp} \rangle \mid \langle \text{SpecTemp} \rangle \\
\langle \text{SpecTemp} \rangle &::= \langle \text{SpecTemp} \rangle \text{ Since } \langle \text{SpecUnary} \rangle \mid \langle \text{SpecTemp} \rangle \text{ Until } \langle \text{SpecUnary} \rangle \mid \langle \text{SpecUnary} \rangle \\
\langle \text{SpecUnary} \rangle &::= ! \langle \text{SpecUnary} \rangle \mid \text{Previously } \langle \text{SpecUnary} \rangle \mid \text{Once } \langle \text{SpecUnary} \rangle \mid \text{Historically } \langle \text{SpecUnary} \rangle \\
&\mid \text{Next } \langle \text{SpecUnary} \rangle \mid \text{Eventually } \langle \text{SpecUnary} \rangle \mid \text{Always } \langle \text{SpecUnary} \rangle \\
&\mid \text{Forall } \langle \text{Ident} \rangle \langle \text{DomainOpt} \rangle . \langle \text{Spec} \rangle \mid \text{Exists } \langle \text{Ident} \rangle \langle \text{DomainOpt} \rangle . \langle \text{Spec} \rangle \\
&\mid \langle \text{BExp} \rangle \mid (\langle \text{Spec} \rangle) \\
\langle \text{DomainOpt} \rangle &::= \epsilon \mid \text{in } \langle \text{Domain} \rangle \\
\langle \text{Domain} \rangle &::= \langle \text{Interval} \rangle \mid \langle \text{Ident} \rangle \mid \langle \text{SetLiteral} \rangle \mid \langle \text{Type} \rangle \\
\langle \text{Interval} \rangle &::= [\langle \text{AExp} \rangle, \langle \text{AExp} \rangle] \mid [\langle \text{AExp} \rangle, \langle \text{AExp} \rangle] \\
\langle \text{SetLiteral} \rangle &::= \{ \langle \text{AExpList} \rangle \} \\
\langle \text{AExpList} \rangle &::= \langle \text{AExp} \rangle (, \langle \text{AExp} \rangle)^*
\end{aligned}$$

1.6 Statements, Functions, and Programs

Top-Level Statements

$$\begin{aligned}
\langle \text{StmtSeq} \rangle &::= \langle \text{Stmt} \rangle^* \\
\langle \text{Stmt} \rangle &::= \langle \text{TermStmt} \rangle \mid \langle \text{IfStmt} \rangle \mid \langle \text{WhileStmt} \rangle \mid \langle \text{AssertStmt} \rangle \mid ; \\
\langle \text{TermStmt} \rangle &::= \langle \text{Decl} \rangle ; \mid \langle \text{Assign} \rangle ; \mid \langle \text{ListAssign} \rangle ; \mid \langle \text{FuncCall} \rangle ; \mid \langle \text{IOStmt} \rangle ; \mid \text{pass} ; \\
\langle \text{IfStmt} \rangle &::= \text{if } \langle \text{BExp} \rangle \text{ then } \langle \text{StmtSeq} \rangle \text{ else } \langle \text{StmtSeq} \rangle \text{ fi} \\
&\mid \text{if } \langle \text{BExp} \rangle \text{ then } \langle \text{StmtSeq} \rangle \text{ fi} \\
\langle \text{WhileStmt} \rangle &::= \text{while } \langle \text{BExp} \rangle \text{ do } \langle \text{StmtSeq} \rangle \text{ od} \\
\langle \text{AssertStmt} \rangle &::= \{ \langle \text{Spec} \rangle \} ; \\
\langle \text{Decl} \rangle &::= \langle \text{Ident} \rangle : \langle \text{Type} \rangle \\
\langle \text{Assign} \rangle &::= \langle \text{Ident} \rangle := \langle \text{AExp} \rangle \\
\langle \text{ListAssign} \rangle &::= \langle \text{IndexedVal} \rangle := \langle \text{AExp} \rangle \\
\langle \text{IOStmt} \rangle &::= \text{read}() \mid \text{write } \langle \text{Ident} \rangle \mid \text{writeln } \langle \text{Ident} \rangle
\end{aligned}$$

²This approach might lead to the generation of broken IR code as a non-valid specification is accepted by the parser. However, in the final design, IR generation is planned to be in memory (i.e. no IR code file to be generated). Thus, in the final implementation, after the generation of IR code of the input program, all specifications will be parsed in the memory with position information before the beginning of execution. This way, a compiler error can still be thrown for invalid specification statements.

Functions

$$\begin{aligned}
\langle \text{FuncDecl} \rangle &::= \text{function } \langle \text{Ident} \rangle (\langle \text{ParamListOpt} \rangle) \rightarrow \langle \text{FuncType} \rangle :: \langle \text{FuncStmtSeq} \rangle \text{ end} \\
\langle \text{ParamListOpt} \rangle &::= \epsilon \mid \langle \text{ParamList} \rangle \\
\langle \text{ParamList} \rangle &::= \langle \text{Param} \rangle (, \langle \text{Param} \rangle)^* \\
\langle \text{Param} \rangle &::= \langle \text{Ident} \rangle : \langle \text{FuncType} \rangle \\
\langle \text{FuncStmtSeq} \rangle &::= \langle \text{FuncStmt} \rangle^*{}^3 \\
\langle \text{FuncStmt} \rangle &::= \langle \text{FuncTermStmt} \rangle \mid \langle \text{FuncIfStmt} \rangle \mid \langle \text{FuncWhileStmt} \rangle \mid \langle \text{AssertStmt} \rangle \mid ; \\
\langle \text{FuncTermStmt} \rangle &::= \langle \text{Decl} \rangle ; \mid \langle \text{Assign} \rangle ; \mid \langle \text{ListAssign} \rangle ; \mid \langle \text{FuncCall} \rangle ; \mid \text{pass} ; \mid \langle \text{ReturnStmt} \rangle ; \\
\langle \text{FuncIfStmt} \rangle &::= \text{if } \langle \text{BExp} \rangle \text{ then } \langle \text{FuncStmtSeq} \rangle \text{ else } \langle \text{FuncStmtSeq} \rangle \text{ fi} \\
&\quad \mid \text{if } \langle \text{BExp} \rangle \text{ then } \langle \text{FuncStmtSeq} \rangle \text{ fi} \\
\langle \text{FuncType} \rangle &::= \langle \text{Type} \rangle \mid \text{List} [\langle \text{Type} \rangle, \langle \text{IntLit} \rangle] \\
\langle \text{FuncWhileStmt} \rangle &::= \text{while } \langle \text{BExp} \rangle \text{ do } \langle \text{FuncStmtSeq} \rangle \text{ od} \\
\langle \text{ReturnStmt} \rangle &::= \text{return } \langle \text{AExp} \rangle
\end{aligned}$$

Return Rule Although not explicitly stated in its grammar, every possible execution `SelVeri` function body must end with a `return` statement. Mostly, our operational semantics will ensure that. However, if the user does not put a `return` statement at the end of the function body; instead of throwing a compilation error, the compiler automatically puts one, returning the default value of the functions return-type. Additionally, as our syntax suggests, a `return` statement can occur only in a function body. These facts will play an important role in our correctness of compilation proofs, therefore we simply name them as the *return rule*.

Function Calls

$$\begin{aligned}
\langle \text{FuncCall} \rangle &::= \text{call} \langle \text{Ident} \rangle (\langle \text{ArgListOpt} \rangle) \\
\langle \text{ArgListOpt} \rangle &::= \epsilon \mid \langle \text{ArgList} \rangle \\
\langle \text{ArgList} \rangle &::= \langle \text{AExp} \rangle (, \langle \text{AExp} \rangle)^*
\end{aligned}$$

Programs

$$\langle \text{Program} \rangle ::= (\langle \text{FuncDecl} \rangle)^* \langle \text{StmtSeq} \rangle$$

Semicolon Discipline Declarations, assignments, list assignments, function-call statements, `pass`, and `return` must end with a semicolon. Statements of the form `if ... fi` and `while ... od` must not be followed by a semicolon.

2 Semantics

2.1 Important notions for formalizing SelVeri

2.1.1 Types

Let **Type** be the set of all possible types. For the current implementation of **SelVeri**, an element of **Type** can be either **Int**, **Float** or a **List** of any dimension, including one of the former two, which can be interpreted as a dependent type. More formally:

- **Int** : **Type**
- **Float** : **Type**
- **List** : $(\Pi t : \mathbf{BasicType} . (\Pi d : \mathbb{N}^+ . (\Pi \vec{n} : \mathbb{N}^d . \mathbf{Type})))$

where $\mathbf{BasicType} := \{\mathbf{Int}, \mathbf{Float}\}$. Similarly, we define $\mathbf{ListType} := \mathbf{Type} \setminus \mathbf{BasicType}$

For convenience, we define the projection function $\pi_d : \mathbb{N}^d \rightarrow \mathbb{N}^{d-1}$:

$$\pi_d : (n_1, \dots, n_d) \mapsto (n_2, \dots, n_d)$$

and throughout this text, we refer the i th component of $\vec{n}$ as n_i .

2.1.2 Immediates

Let **Imm** be the set of syntactic immediate values supported by **SelVeri**, e.g. 5 or 3.14, and **Var** be the infinite set of all syntactic variables. For each type, we need an evaluation function that translates syntactic immediates into the corresponding semantic values.

$$\begin{aligned} \mathcal{I} : \mathbf{Imm} \cap \mathbf{Int} &\rightarrow \mathbb{Z} \\ \mathcal{F} : \mathbf{Imm} \cap \mathbf{Float} &\rightarrow \mathbb{R} \\ \mathcal{L}_{\mathbf{List}[t, d, \vec{n}]} : \mathbf{Imm} \cap \mathbf{List}[t, d, \vec{n}] &\rightarrow A^{n_1} \end{aligned}$$

where A is the codomain of $\mathcal{L}_{\mathbf{List}[t, d-1, \pi_d(\vec{n})]}$. The reader should notice that:

- $\mathbf{List}[t, 0, ()] = t$
- $\mathcal{L}_{\mathbf{Int}} = \mathcal{I}$ and $\mathcal{L}_{\mathbf{Float}} = \mathcal{F}$

For simplicity, we also define $\mathcal{V} := \bigcup_{t \in \mathbf{Type}} \mathcal{L}_t$

2.1.3 Scope

Let **Scope** be the set of all scopes and **AExp** be the set of all arithmetic expressions. Each scope is a representation of 'known' variables in a given block of statements including the variables declared until program execution reaches the corresponding block. Formally,

$$\forall s \in \mathbf{Scope} . s : \mathbf{AExp} \rightarrow \mathbf{Type}$$

Given any scope, s , immediates are mapped to the corresponding types; e.g. $s(5) = \mathbf{Int}$, $s([1, 2, 3]) = \mathbf{List}[\mathbf{Int}, 1, 3]$, as expected. Additionally, given any non-atomic arithmetic expression a , we have

$$s(a) = \begin{cases} \mathbf{Int} & \text{if } s(a') = \mathbf{Int} \text{ for all subterms } a' \text{ of } a \\ \mathbf{Float} & \text{if } s(a') = \mathbf{Float} \text{ for some subterm } a' \text{ of } a \end{cases}$$

Note that a composite arithmetic expression cannot have a non-basic type in *SelVeri* as we do not allow a list literal to interact with arithmetic operators.

Initially, each scope starts its lifetime as an empty scope, denoted as $\tilde{s}$,

$$\forall x \in \mathbf{Var} . \tilde{s}(x) = \mathbf{Void}$$

where $\mathbf{Void} : \mathbf{Type}$ is the empty type.

To simplify the formalization of list-types, given any scope s , we enforce the following implication:

$$s(x) = \mathbf{List}[t, d, \vec{n}] \implies \forall i \in \{0, \dots, n-1\} . s(x[i]) = \mathbf{List}[t, d-1, \pi_d(\vec{n})]$$

We call this implication as *recursive declaration principle*. As **SelVeri** syntax does not allow identifiers including [or] characters, it is not possible to create any scope contradicting the recursive declaration.

Moreover, each scope comes along with a (possibly empty) **parent_scope**. If s is a scope, we denote its immediate child as s_c . In the **SelVeri** implementation, a variable is searched in the **parent_scope**, if it cannot be found in the current scope. When a variable is reached by a parent scope, then a change to its type (which is only possible for special variable **retvar**, see section 2.1.5) is made directly to the parent scope. However, for simplicity, we omit this special case in our configurations.

2.1.4 Program states

Let **State** be the set of all possible program states and **Val**⁴ be the set of all semantic values. Each state is in fact a map from **Var** and **Scope** to **Val**, representing the value of a given variable in the specified scope. Formally,

$$\forall \sigma \in \mathbf{State} . \sigma : \mathbf{Var} \times \mathbf{Scope} \rightarrow \mathbf{Val}$$

Each program starts with an empty state, $\tilde{\sigma}$ in which each declared variable has the default value of this type, e.g. 0 for **Int**, 0.0 for **Float** and [0, 0, 0] for **List[Int, 1, 3]**. If the specific type is unknown in a given context, the default value of the corresponding type is denoted as **0**.

Also, similar to scopes, program states also have parents. A child program state of σ is denoted as σ_c . When a variable is reached by a parent scope, then its value must be reached by the parent program state. The change must be directly made in the parent program state, in case of an assignment. However, for simplicity, we omit this special case in our configurations.

Additionally, each program state carries an **error_flag** that is set if and only if an error occurs during the execution of the program. A cause for this can be using a variable before declaring it. In order to embed this misuse as an error into our semantics, we add the following rule to our program states:

$$\forall \sigma \in \mathbf{State} . \sigma(x, s) = \perp \quad \text{if } s(x) = \mathbf{Void}.$$

Before moving to the next section, reader should be aware of an abuse of notation we use for convenient representation of erroneous states:

- σ represents a program state without any errors.
- $\hat{\sigma}$ represents a program state with an error.
- $\underline{\sigma}$ represents any program state (may be error-free or erroneous).

⁴The reader can notice that $\text{ran}\mathcal{I}$, $\text{ran}\mathcal{F}$ and $\text{ran}\mathcal{L}$ are all subsets of **Val**.

2.1.5 Functions

Let **FuncName** be the infinite set of all valid **SelVeri** function names. To store the information of all function definitions in a given program, we introduce the concept of the *function environment*, denoted as Γ :

$$\Gamma : \mathbf{FuncName} \rightarrow (\mathbf{Var} \times \mathbf{Type})^n \times \mathbf{Type} \times \mathbf{Code}$$

where n is the arity of the input function and **Code** is the set of all syntactically-correct **SelVeri** programs. During the compilation stage, compiler populates Γ with a single-pass through the function definitions.

For example, if one defines the function **f** as in the following way:

```
function f(x:Int, y:Float, z:List[Float,1,2]) -> Int :: Sf end
```

where S_f is some function statement sequence, then

$$\Gamma(f) = ((x, \text{Int}), (y, \text{Float}), (z, \text{List}[\text{Float}, 1, 2])), \text{Int}, S_f)$$

On the other hand, if **g** is an undefined function, then $\Gamma(g) = \emptyset$.

Moreover, we introduce a special, dynamically-typed variable **retvar** which holds the return-value of the last called function. Initially, its type is **Void** and its value is $\perp$. **retvar** is a reserved keyword in **SelVeri**, meaning that no user-defined identifier can have this name. An example usage would be:

```
w:Int; call f(5, 3.14, [42, 42]); w := retvar;
```

Assuming that, with the inputs above, execution of the body of **f** does not result in an error state, our operational semantics will ensure that variable **w** holds the result of the computation of **f** (the value returned from **f** to be exact).

2.1.6 List sizes

SelVeri supports multi-dimensional lists and to be able to formalize them correctly we define the following helper function $\text{size} : \mathbf{Type} \rightarrow \mathbb{N}$:

$$\text{size}(t) = \begin{cases} 0 & \text{if } t \in \mathbf{BasicType} \\ \prod_{i=1}^d n_i & \text{if } t = \mathbf{List}[t', d, \vec{n}] \text{ for some } t' \in \mathbf{BasicType} \end{cases}$$

In a type declaration like $\mathbf{List}[t, d, \vec{n}]$, definition of **List** type enforces $t \in \mathbf{BasicType}$, so above definition covers all possible t values.

2.1.7 I/O Operations

SelVeri programs can interact with the input and output streams. This is necessary due to the philosophy of **SelVeri**, since verification requires reasoning over programs that depend on external inputs rather than fixed constants. These interactions are done utilizing tools available to the abstract machine⁵.

Write Operations. Write operations behave as expected. The string representation of an arithmetic expression is printed after evaluating it under the current state and scope. Formally, for an expression a , the abstract machine outputs $\mathcal{A}(a, \sigma, s)$.

⁵Refer to SelVerIR document for details of the abstract machine.

Read as an Arithmetic Expression. Unlike the `write` operation, `read` is not treated as a statement, and it is evaluated as an arithmetic expression. This allows it to be composed naturally inside larger expressions, such as:

$$x := \text{read}() + (\text{read}() * 2)$$

To support this, we define all possible sequences of input streams as **InputStream**:

$$\iota \in \mathbf{InputStream} \implies \exists n \in \mathbb{N}. \iota = [v_0, v_1, v_2, \dots, v_n] \in \mathbf{Val}^n$$

which represents the sequence of values provided by the environment.

⁶Here, n represents the number of inputs provided by the user through the execution of the program. Normally it is equal to the number of occurrences of the `read()` statement.

2.2 Semantics of arithmetic expressions

We define the arithmetic valuation function $\mathcal{A}$, representing the semantics of the arithmetic expressions in **SelVeri**.

$$\mathcal{A} : \mathbf{AExp} \times \mathbf{State} \times \mathbf{Scope} \times \mathbf{InputStream} \rightarrow \mathbf{Val}$$

$\mathcal{A}(x, \sigma, s, \iota) = \mathcal{I}(x)$	if $x \in \mathbf{Imm} \wedge s(x) = \mathbf{Int}$
$\mathcal{A}(x, \sigma, s, \iota) = \mathcal{F}(x)$	if $x \in \mathbf{Imm} \wedge s(x) = \mathbf{Float}$
$\mathcal{A}(x, \sigma, s, \iota) = \text{size}(\mathbf{List}[t, d, \vec{n}]) : \mathcal{L}_{\mathbf{List}[t, d, \vec{n}]}(x)$	if $x \in \mathbf{Imm} \wedge s(x) = \mathbf{List}[t, d, \vec{n}]$
$\mathcal{A}(x, \sigma, s, \iota) = \sigma(x, s)$	if $x \in \mathbf{Var}$
$\mathcal{A}(lst[i_1] \dots [i_m], \sigma, s, \iota) = \mathcal{A}(lst[\mathcal{A}(i_1, \sigma, s, \iota)] \dots [\mathcal{A}(i_m, \sigma, s, \iota)], \sigma, s, \iota)$	if $lst \in \mathbf{Var}$
$\mathcal{A}(\text{len}(x), \sigma, s, \iota) = \begin{cases} 0 & \text{if } x \in \mathbf{Var} \wedge s(x) \in \mathbf{BasicType} \\ \mathcal{A}(_x.\text{len}.1, \sigma, s, \iota) & \text{if } x \in \mathbf{Var} \wedge s(x) \notin \{\mathbf{Void}, \mathbf{Int}, \mathbf{Float}\} \\ \perp & \text{otherwise} \end{cases}$	
$\mathcal{A}(a_1 + a_2, \sigma, s, \iota) = \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) + \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \perp \vee \mathcal{A}(a_2, \sigma, s, \iota) = \perp \end{cases}$	
$\mathcal{A}(a_1 * a_2, \sigma, s, \iota) = \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) \cdot \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \perp \vee \mathcal{A}(a_2, \sigma, s, \iota) = \perp \end{cases}$	
$\mathcal{A}(a_1 - a_2, \sigma, s, \iota) = \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) - \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \perp \vee \mathcal{A}(a_2, \sigma, s, \iota) = \perp \end{cases}$	
$\mathcal{A}(a_1 / a_2, \sigma, s, \iota) = \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) / \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_2, \sigma, s, \iota) \neq 0 \wedge \mathcal{A}(a_1, \sigma, s, \iota) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_2, \sigma, s, \iota) = 0 \vee \mathcal{A}(a_1, \sigma, s, \iota) = \perp \vee \mathcal{A}(a_2, \sigma, s, \iota) = \perp \end{cases}$	
$\mathcal{A}(\text{read}(), \sigma, s, i : \iota) = i$	if $i \in \mathbf{Imm} \wedge s(x) \in \mathbf{BasicType}$

Table 1: The semantics of arithmetic expressions

Although not explicitly stated in the table above, given any binary operation, operators are required to be of the same type (except for the cases of implicit coercion⁷ handled by the compiler) and can only be of *basic types*⁸. Otherwise, the result of the evaluation is $\perp$. The same rules apply to the next section (boolean expressions) as well.

⁶If (a_1/a_2) has no subterm of type **Float**, then the $/$ symbol represents the integer division; otherwise, it represents the division of the field $\mathbb{R}$.

⁷An example is multiplying an integer with a float. In such cases, compiler converts the integer to a float with the same value. Conversion of integers to floats is the only possibility for an implicit coercion, as for the current implementation of **SelVeri**.

⁸The only basic types are **Int** and **Float** for the current implementation of **SelVeri**.

2.3 Semantics of boolean expressions

Let **BExp** be the set of all boolean expressions. We define the arithmetic valuation function $\mathcal{B}$, representing the semantics of the boolean expressions in **SelfVeri**.

$$\mathcal{B} : \mathbf{BExp} \times \mathbf{State} \times \mathbf{Scope} \rightarrow \{0, 1, \perp\}$$

$\mathcal{B}(\text{true}, \sigma, s)$	$= 1$
$\mathcal{B}(\text{false}, \sigma, s)$	$= 0$
$\mathcal{B}(a, \sigma, s)$ ⁹	$= \begin{cases} 1 & \text{if } \mathcal{A}(a, \sigma, s) \neq \mathbf{0} \\ 0 & \text{if } \mathcal{A}(a, \sigma, s) = \mathbf{0} \\ \perp & \text{if } \mathcal{A}(a, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(a_1 = a_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \mathcal{A}(a_2, \sigma, s) \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(a_1 \leq a_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) \leq \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) > \mathcal{A}(a_2, \sigma, s) \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(\text{not } b, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{B}(b, \sigma, s) = 0 \\ 0 & \text{if } \mathcal{B}(b, \sigma, s) = 1 \\ \perp & \text{if } \mathcal{B}(b, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(b_1 \text{ and } b_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{B}(b_1, \sigma, s) = 1 \text{ and } \mathcal{B}(b_2, \sigma, s) = 1 \\ 0 & \text{if } \mathcal{B}(b_1, \sigma, s) = 0 \text{ or } \mathcal{B}(b_2, \sigma, s) = 0 \\ \perp & \text{if } \mathcal{B}(b_1, \sigma, s) = \perp \text{ or } \mathcal{B}(b_2, \sigma, s) = \perp \end{cases}$

Table 2: Semantics of boolean expressions

⁹To have this valuation rule, we need $\mathbf{AExp} \subseteq \mathbf{BExp}$ which is the case for **SelfVeri**. What this rule means is, every arithmetic expression can also be interpreted as a boolean expression depending on the context and its truth value is determined by an equality to zero check, similar to the C programming language.

2.4 Structural operational semantics of the statements

$[\text{decl}_{\text{os}}]$	$\langle x : t, \sigma, s \rangle \rightarrow \langle \sigma[x \mapsto \mathbf{0}], s[x \mapsto t] \rangle \quad \text{if } t \in \mathbf{BasicType} \wedge s(x) = \mathbf{Void}$
$[\text{decl}_{\text{os}}^{\text{list}}]$	$\langle \text{lst} : \mathbf{List}[t, d, \vec{a}], \sigma, s \rangle \rightarrow \langle \sigma[\text{lst} \mapsto \mathbf{0}, \text{lst_len_i} \mapsto \mathcal{A}(a_i, \sigma, s)], s[\text{lst} \mapsto \mathbf{List}[t, d, \mathcal{A}(a_i, \sigma, s)], \text{lst_len_i} \mapsto \mathbf{Int}] \rangle$ for $i \in \{1, \dots, d\}$ and if $t \in \mathbf{BasicType} \vee s(x) = \mathbf{Void}$
$[\text{decl}_{\text{os}}^{\perp}]$	$\langle x : t, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } t \notin \mathbf{Type} \vee s(x) \neq \mathbf{Void}$
$[\text{ass}_{\text{os}}]$	$\langle x := a, \sigma, s \rangle \rightarrow \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad \text{if } \mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in \text{ran}\mathcal{L}_{s(x)}$
$[\text{ass}_{\text{os}}^{\text{list}}]$	$\langle \text{lst}[i_1] \dots [i_m] := a, \sigma, s \rangle \rightarrow \sigma[\text{lst}[\mathcal{A}(i_1, \sigma, s)] \dots [\mathcal{A}(i_m, \sigma, s)] \mapsto \mathcal{A}(a, \sigma, s)]$ if $\mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in \text{ran}\mathcal{L}_{s(\text{lst}[i_1] \dots [i_m])}$
$[\text{ass}_{\text{os}}^{\perp}]$	$\langle x := a, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } \mathcal{A}(a, \sigma, s) = \perp \vee \mathcal{A}(a, \sigma, s) \notin \text{ran}\mathcal{L}_{s(x)}$
$[\text{pass}_{\text{os}}]$	$\langle \text{pass}, \sigma, s \rangle \rightarrow \sigma$
$[\text{comp}_{\text{os}}^1]$	$\frac{\langle S_1, \sigma, s \rangle \rightarrow \langle S'_1, \sigma', s' \rangle}{\langle S_1; S_2, \sigma, s \rangle \rightarrow \langle S'_1; S_2, \sigma', s' \rangle}$
$[\text{comp}_{\text{os}}^2]$	$\frac{\langle S_1, \sigma, s \rangle \rightarrow \sigma'}{\langle S_1; S_2, \sigma, s \rangle \rightarrow \langle S_2, \sigma', s \rangle}$
$[\text{scope}_{\text{os}}]$	$\langle \text{endscope}, \sigma_c, s_c \rangle \rightarrow \langle \sigma, s \rangle \quad \text{where } \sigma, s \text{ are parents of } \sigma_c, s_c$
$[\text{if}_{\text{os}}^1]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \rightarrow \langle S_1; \text{endscope}, \tilde{\sigma}_c, \tilde{s}_c \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{if}_{\text{os}}^0]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \rightarrow \langle S_2; \text{endscope}, \tilde{\sigma}_c, \tilde{s}_c \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{if}_{\text{os}}^{\perp}]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{while}_{\text{os}}^1]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \langle S; \text{endscope}; \text{while } b \text{ do } S \text{ od}, \tilde{\sigma}_c, \tilde{s}_c \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{while}_{\text{os}}^0]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \sigma \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{while}_{\text{os}}^{\perp}]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{call}_{\text{os}}]$	$\langle \text{call } f(a_1, \dots, a_n), \sigma, s \rangle \rightarrow \langle x_1 : t_1; x_1 := v_1; \dots; x_n : t_n; x_n := v_n; S_f, \tilde{\sigma}', \tilde{s}' \rangle$ if $t_i = s(x_i)$ for $i = 1, 2, \dots, n$ where $\Gamma(f) = (((x'_1, t_1), \dots, (x'_n, t_n)), t', S_f)$ and $v_j \in \mathbf{Imm} \wedge \mathcal{A}(v_j, \tilde{\sigma}', \tilde{s}') = \mathcal{A}(x_j, \sigma, s)$ for $j = 1, 2, \dots, n$
$[\text{ret}_{\text{os}}]$	$\langle \text{return } a, \sigma', s' \rangle \rightarrow \langle \sigma[\text{retvar} \mapsto \mathcal{A}(a, \sigma', s')], s[\text{retvar} \mapsto s'(a)] \rangle$ if the return-type of the function called is equal to $s'(a)$ where σ and s are of returning function's caller.
$[\text{write}_{\text{os}}]$	$\langle \text{write}(x), \sigma, s \rangle \rightarrow \langle \sigma, s \rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$
$[\text{writeln}_{\text{os}}]$	$\langle \text{writeln}(x), \sigma, s \rangle \rightarrow \langle \sigma, s \rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$ and a newline
$[\text{err}_{\text{os}}]$	$\langle S, \hat{\sigma}, s \rangle \rightarrow \hat{\sigma}$

Table 3: Small-step operational semantics for statements

Note that not all possible configurations are included in the table above. In any case which is absent from the table, program state is transitioned into $\hat{\sigma}$.

Moreover, for each if-rule, an if-statement without the **else**-part, namely a statement of the form "**if** b **then** S **fi**" can be treated as "**if** b **then** S **else pass fi**"